



STUDENT'S NAME: SAIDA NAMSONGE JUMA

ADMISSION NUMBER: BSCCS/2024/42912

COURSE: BACHELORS IN COMPUTER SCIENCE

UNIT CODE: BIT 4133

UNIT NAME: NATURAL LANGUAGE PROCESSING WITH DEEP
LEARNING

LECTURER NAME: MICHAEL NYORO

ACADEMIC YEAR: 2025/2026 MAY- AUGUST

PROJECT TITLE:
AI BASED SENTIMENT ANALYSIS SYSTEM USING PYTHON
AND NLTK.

TECHNOLOGIES USED

PYTHON

NLTK

SCIKIT-LEARN

GOOGLE COLAB

JUPYTER NOTEBOOK

GITHUB REPOSITORY

<https://github.com/Saida-Namsonge/NLP-logbook-project.git>

TABLE OF CONTENTS

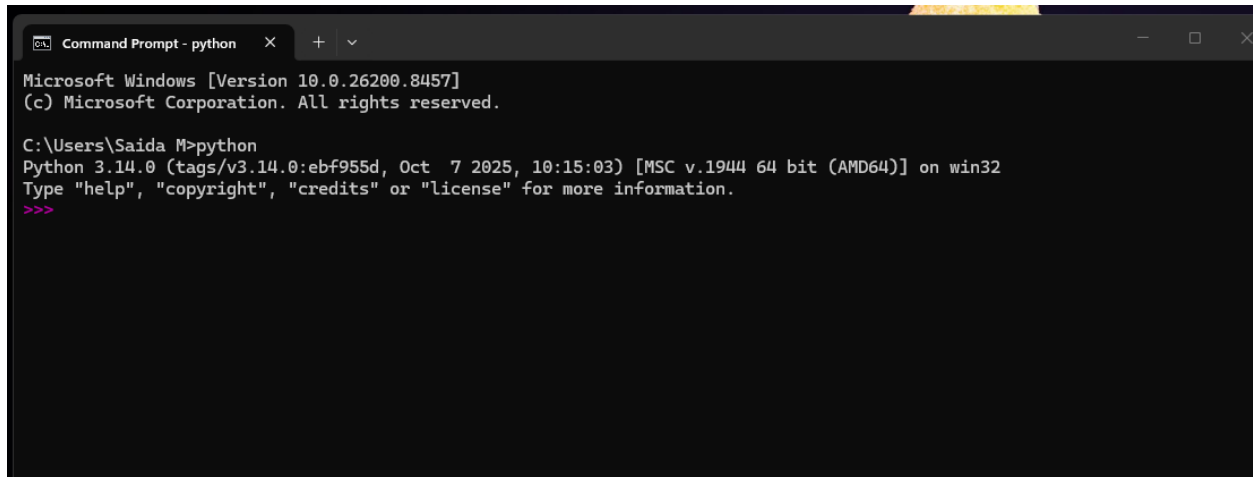
WEEK 1: INTRODUCTION TO NLP AND CONCEPTS AND APPLICATIONS.....	2
TITLE: Introduction to NLP concepts and text processing	3
Fig 1: Python Installation.	3
Fig 2: Google colab setup	3
Fig 3: NLTK Installation	3
Fig 4: Tokenization Output.....	4
Fig 5: Stopwords Removal Output	5
Fig 6: NLP Application Research Evidence (Spam Filtering)	5
REFLECTION.....	6
WEEK 2: N-Gram Models and POS Tagging	6
TITLE: Language Modelling and Sentence Structure Analysis.	7
Fig 1: Bigram or Trigram Output	7
Fig 1.1: Bigram	7
fig 1.2: Trigram.....	7
Fig 2: POS Tagging Examples	7
Fig 3: Python code for POS Tagging.....	8
Fig 4: Language Prediction Example	8
Fig 5: Sentence Analysis Output	9
REFLECTION.....	10
WEEK 3: HIDDEN MARKOV MODELS AND SEQUENCE LABELING.....	11
TITLE: Sequence Prediction and Hidden State Analysis	12
Fig 1: Sequence Labeling Code	12
Fig 2: Hidden Markov Model Output.....	12
Fig 3: Named Entity Recognition Example.....	13
Fig 4: Sentence Labelling Results	13
Fig 5: Dataset used for training or testing	14
REFLECTION.....	14
WEEK 4: SYNTACTIC AND SEMANTIC ANALYSIS.....	15
Example:.....	16

Example:.....	16
PRACTICAL TASK 2	17
CLASS ASSIGNMENT.....	18
Assignment 1	18
Sentence 1: “The administrator updated student records”.....	18
Sentence 2: “Machine learning improves language processing”	19
Assignment 2.....	20
PRACTICAL TASK 1	22
Complete NLP Pipeline.....	22
Objective.....	22
Build a simple NLP processing pipeline.....	22
PRACTICAL TASK 2.....	22
Mini Chatbot Simulation	23
Objective.....	23
Create a simple rule-based chatbot.	23
WEEK 5 MINI PROJECT	23
Project Title	24
Student Academic Assistant Chatbot	24
Challenge:	24
REVISION AREAS	24

WEEK 1: INTRODUCTION TO NLP AND CONCEPTS AND APPLICATIONS

TITLE: Introduction to NLP concepts and text processing

Fig 1: Python Installation.



```
Command Prompt - python
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Saida M>python
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Fig 1 shows that python version 3.14 was successfully installed to provide the programming environment required for NLP experiments.

Fig 2: Google colab setup

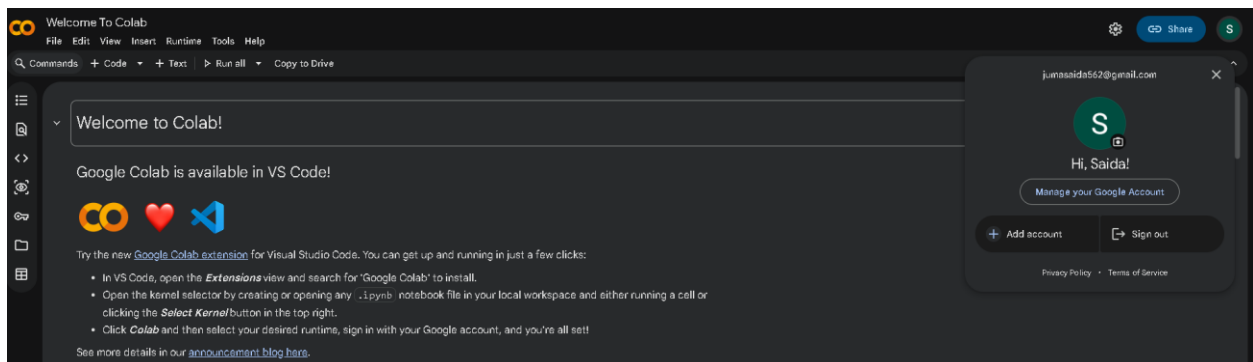


Fig 2 shows the complete setup of google colab that allows one to write, run and share python codes directly in the web browser.

Fig 3: NLTK Installation

```
[1] 8s ▶ !pip install nltk
✓ 8s
... Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.4.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
```

```
[3] 41s ▶ import nltk
✓ 41s
nltk.download('all')
... [nltk_data] Downloading collection 'all'
[nltk_data]
[nltk_data] | Downloading package abc to /root/nltk_data...
[nltk_data] | Unzipping corpora/abc.zip.
[nltk_data] | Downloading package alpino to /root/nltk_data...
[nltk_data] | Unzipping corpora/alpino.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger_eng to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping
[nltk_data] | taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger_ru to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping
[nltk_data] | taggers/averaged_perceptron_tagger_ru.zip.
[nltk_data] | Downloading package averaged_perceptron_tagger_rus to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping
[nltk_data] | taggers/averaged_perceptron_tagger_rus.zip.
[nltk_data] | Downloading package basque_grammars to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping grammars/basque_grammars.zip.
[nltk_data] | Downloading package bcp47 to /root/nltk_data...
[nltk_data] | Downloading package biocreative_ppi to
[nltk_data] | /root/nltk_data...
[nltk_data] | Unzipping corpora/biocreative_ppi.zip.
[nltk_data] | Downloading package bllip_wsj_no_aux to
[nltk_data] | /root/nltk_data...
```

Fig 3 shows the installation of NLTK.

NLTK is a Python's API library, and it can perform a variety of operations on textual data such as classification, tokenization, stemming, tagging, semantic reasoning, etc.

Fig 4: Tokenization Output

```
[5]
✓ Os # Tokenization using NLTK
from nltk import word_tokenize, sent_tokenize
sent = "GeeksforGeeks is a great learning platform.\nIt is one of the best for Computer Science students."
print(word_tokenize(sent))
print(sent_tokenize(sent))

... ['GeeksforGeeks', 'is', 'a', 'great', 'learning', 'platform.It', 'is', 'one', 'of', 'the', 'best', 'for', 'Computer', 'Science', 'students', '.']
['GeeksforGeeks is a great learning platform.It is one of the best for Computer Science students.']
```

Fig 4 shows a tokenization output.

Tokenization refers to the breakdown of text into smaller units. It splits paragraphs into sentences and sentences into words. There are two types of tokenization as illustrated above that is word tokenization and sentence tokenization.

Fig 5: Stopwords Removal Output

```
[5]
✓ Os import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('stopwords')
nltk.download('punkt')

# Sample text
text = "This is a sample sentence showing stopword removal."

# Get English stopwords and tokenize
stop_words = set(stopwords.words('english'))
tokens = word_tokenize(text.lower())

# Remove stopwords
filtered_tokens = [word for word in tokens if word not in stop_words]

print("Original:", tokens)
print("Filtered:", filtered_tokens)

... Original: ['this', 'is', 'a', 'sample', 'sentence', 'showing', 'stopword', 'removal', '.']
Filtered: ['sample', 'sentence', 'showing', 'stopword', 'removal', '.']
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Fig 5 shows stopwords removal

Natural language processing tasks often involve filtering out commonly occurring words that provide no or very little semantic value to text analysis. These words are known as stopwords include articles, prepositions and pronouns like "the", "and" "is" and "in". While they seem insignificant, proper stopword handling can impact the performance and accuracy of NLP applications.

Fig 6: NLP Application Research Evidence (Spam Filtering)

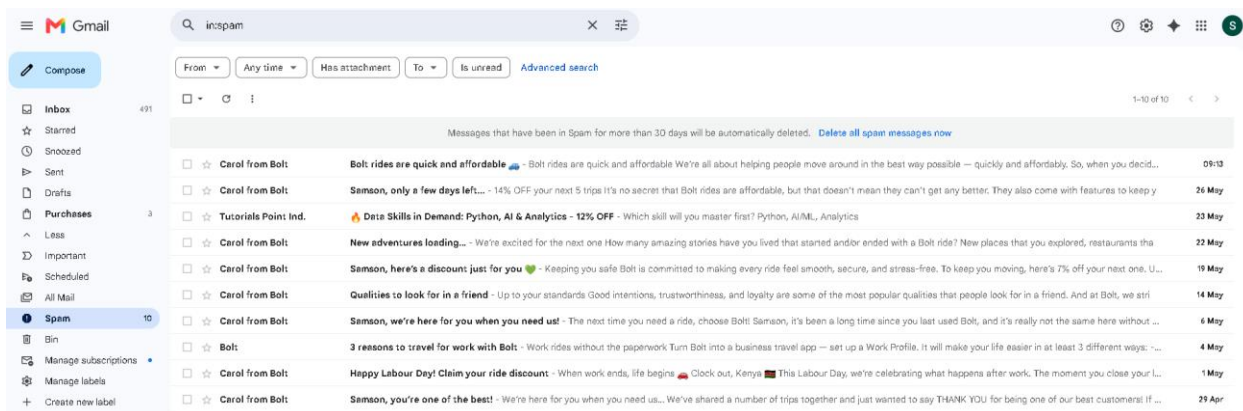


Fig 6 shows research evidence from spam messages in Gmail

Spam filtering systems use NLP techniques to analyze email text and classify messages as spam or legitimate emails.

REFLECTION

During this practical session, I learned how NLP systems preprocess text before machine learning analysis. I successfully installed Python and NLTK. I also successfully setup Google Colab then implemented tokenization and stopwords removal. The outputs demonstrated how text is prepared for NLP tasks such as sentiment analysis and chatbot development. I also explored real-world NLP applications such as ChatGPT and spam filtering in Gmail and understood their importance in AI systems. I learned that Tokenization is the breakdown of texts into smaller units, and stopwords removal is the filtering of commonly occurring words that have little semantic value.

WEEK 2: N-Gram Models and POS Tagging

TITLE: Language Modelling and Sentence Structure Analysis.

Fig 1: Bigram or Trigram Output

Fig 1.1: Bigram

```
import nltk
from nltk.util import bigrams
from nltk.tokenize import word_tokenize

# Download tokenizer
nltk.download('punkt')

# Sample text
text = "Natural Language Processing is interesting"

# Tokenize sentence
tokens = word_tokenize(text)

# Generate bigrams
bigram_output = list(bigrams(tokens))

# Display output
print(bigram_output)

... [('Natural', 'Language'), ('Language', 'Processing'), ('Processing', 'is'), ('is', 'interesting')]
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

The code demonstrates the generation of bigrams using the NLTK library. The sentence was tokenized into individual words before pairs of consecutive words were created. Bigram models are important in NLP because they help systems understand word relationships and improve text prediction accuracy in applications such as chatbots and autocomplete systems.

fig 1.2: Trigram

```
import nltk
from nltk.util import trigrams
from nltk.tokenize import word_tokenize

# Download tokenizer
nltk.download('punkt')

# Sample text
text = "Natural Language Processing is interesting"

# Tokenize sentence
tokens = word_tokenize(text)

# Generate trigrams
trigram_output = list(trigrams(tokens))

# Display output
print(trigram_output)

... [('Natural', 'Language', 'Processing'), ('Language', 'Processing', 'is'), ('Processing', 'is', 'interesting')]
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

The trigram model groups three consecutive words together to provide better contextual understanding of language patterns. Trigrams are more powerful than bigrams because they capture more sentence context.

Fig 2: POS Tagging Examples

Example of POS Tagging

Consider the sentence: "The quick brown fox jumps over the lazy dog."

After performing POS Tagging, we get:

- "The" is tagged as determiner (DT)
- "quick" is tagged as adjective (JJ)
- "brown" is tagged as adjective (JJ)
- "fox" is tagged as noun (NN)
- "jumps" is tagged as verb (VBZ)
- "over" is tagged as preposition (IN)
- "the" is tagged as determiner (DT)
- "lazy" is tagged as adjective (JJ)
- "dog" is tagged as noun (NN)

Each word is assigned a tag based on its role in the sentence. For example, "quick" and "brown" are adjectives that describe the noun "fox."

Fig 3: Python code for POS Tagging

```
[30]
✓ 0s
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
text = "NLTK is a powerful library for natural language processing."
words = word_tokenize(text)

pos_tags = pos_tag(words)
print("Original Text:")
print(text)

print("\nPoS Tagging Result:")
for word, pos_tag in pos_tags:
    print(f"{word}: {pos_tag}")

... [nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
Original Text:
NLTK is a powerful library for natural language processing.

PoS Tagging Result:
NLTK: NNP
is: VBZ
a: DT
powerful: JJ
library: NN
for: IN
natural: JJ
language: NN
processing: NN
.: .
```

First we import the NLTK library and download the necessary datasets using `nltk.download()`. Then we store the sentence and tokenize it into words using `word_tokenize(text)`. Then we apply POS tagging to the tokenized words using `pos_tag(words)`. This assigns a part-of-speech to each word. Finally, we print the original sentence and loop through the tagged words to show each word with its POS tag.

Fig 4: Language Prediction Example

```
from collections import defaultdict
from nltk.tokenize import word_tokenize
import nltk

nltk.download('punkt')

text = "I love studying Natural Language Processing"

tokens = word_tokenize(text)

# Build prediction dictionary
prediction = defaultdict(list)

for i in range(len(tokens) - 1):
    prediction[tokens[i]].append(tokens[i + 1])

# Display predictions
for word in prediction:
    print(word, "->", prediction[word])

... I -> ['love']
love -> ['studying']
studying -> ['Natural']
Natural -> ['Language']
Language -> ['Processing']
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

The language prediction experiment demonstrates how NLP systems predict the next word based on previously observed word patterns. The model analyzed sequences of words and generated possible next-word predictions using a simple bigram approach.

Fig 5: Sentence Analysis Output

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk import pos_tag

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

text = "Natural Language Processing is transforming the world of Artificial Intelligence. It is a key area of AI."

sentences = sent_tokenize(text)

for i, sent in enumerate(sentences, start=1):
    print(f"\nSentence {i}: {sent}")

    tokens = word_tokenize(sent)

    tagged = pos_tag(tokens)

    print("\nWord-by-word analysis:")

    for word, tag in tagged:
        print(f"{word:15} -> {tag}")

Sentence 1: Natural Language Processing is transforming the world of Artificial Intelligence.

Word-by-word analysis:
Natural      -> JJ
Language     -> NNP
Processing   -> NNP
is           -> VBZ
transforming -> VBG
the          -> DT
world        -> NN
of           -> IN
Artificial   -> JJ
Intelligence -> NNP
.            -> .

Sentence 2: It is a key area of AI.

Word-by-word analysis:
It           -> PRP
is           -> VBZ
a            -> DT
key          -> JJ
area         -> NN
of           -> IN
AI           -> NNP
.            -> .

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
```

REFLECTION

This exercise helped me understand how NLP techniques break text into meaningful patterns. The bigram and trigram outputs showed how words are linked in sequences, making it easier to predict the next word

based on context. POS tagging helped identify the grammatical role of each word, such as nouns, verbs, and adjectives, which is important for understanding sentence structure. Language prediction demonstrated how models estimate likely next words using probability. Sentence analysis combined these methods to show structure and meaning. I learned that challenges include handling rare words, incorrect tagging, and limited context in short n-grams, which can affect prediction accuracy overall.

WEEK 3: HIDDEN MARKOV MODELS AND SEQUENCE LABELING

TITLE: Sequence Prediction and Hidden State Analysis

Fig 1: Sequence Labeling Code

```
import nltk
import random
from nltk.corpus import treebank
from nltk.tag import hmm

nltk.download('treebank')

tagged_sentences = treebank.tagged_sents()

train_data = tagged_sentences[:3000]
test_data = tagged_sentences[3000:]

trainer = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train(train_data)

sentence = "The president visited Nairobi".split()

print("Sequence Labeling Output:")
print(hmm_model.tag(sentence))

*** [nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data] Package treebank is already up-to-date!
Sequence Labeling Output:
[('The', 'DT'), ('president', 'NN'), ('visited', 'VBD'), ('Nairobi', 'NNP')]
```

Fig 2: Hidden Markov Model Output

```
import nltk
from nltk.corpus import treebank
from nltk.tag import hmm
nltk.download('treebank')
tagged_sentences = treebank.tagged_sents()
print("Total sentences in dataset:", len(tagged_sentences))
train_data = tagged_sentences[:3000]
test_data = tagged_sentences[3000:]
print("Training sentences:", len(train_data))
print("Testing sentences:", len(test_data))
trainer = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train(train_data)
sentence = "The president visited Nairobi".split()
tagged_output = hmm_model.tag(sentence)
for word, tag in tagged_output:
    print(f"{word:15} -> {tag}")
sentence2 = "Natural Language Processing is powerful".split()
print(hmm_model.tag(sentence2))
accuracy = hmm_model.evaluate(test_data)
print("Accuracy:", accuracy)
print(hmm_model.probability(train_data[0]))
```

```
... [nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data] Package treebank is already up-to-date!
Total sentences in dataset: 3914
Training sentences: 3000
Testing sentences: 914
The -> DT
president -> NN
visited -> VBD
Nairobi -> NNP
[('Natural', 'NNP'), ('Language', 'NNP'), ('Processing', 'NNP'), ('is', 'NNP'), ('powerful', 'NNP')]
/tmp/ipykernel_2882/1127482923.py:19: DeprecationWarning:
  Function evaluate() has been deprecated. Use accuracy(gold)
  instead.
  accuracy = hmm_model.evaluate(test_data)
Accuracy: 0.36844377293330455
1.1129179393023502e-52
```

Fig 3: Named Entity Recognition Example

```
from nltk import word_tokenize, pos_tag, ne_chunk
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
text = "Barack Obama was born in Hawaii in 1961."
tokens = word_tokenize(text)
tags = pos_tag(tokens)
entities = ne_chunk(tags)
print(entities)
```

```
... [nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data] /root/nltk_data...
[nltk_data] Package maxent_ne_chunker_tab is already up-to-date!
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data] Package words is already up-to-date!
(S
  (PERSON Barack/NNP)
  (PERSON Obama/NNP)
  was/VBD
  born/VBN
  in/IN
  (GPE Hawaii/NNP)
  in/IN
  1961/CD
  ./.)
```

Fig 4: Sentence Labelling Results

```
import nltk
from nltk.corpus import treebank
from nltk.tag import hmm

nltk.download('treebank')

tagged_sentences = treebank.tagged_sents()

print("Dataset loaded successfully!")
print("Total sentences:", len(tagged_sentences))

train_data = tagged_sentences[:3000]
test_data = tagged_sentences[3000:]

trainer = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train(train_data)

print("\nHMM Model trained successfully!\n")

sentence = "Natural Language Processing is fascinating".split()

print("Labeled Sentence:")
print(hmm_model.tag(sentence))

... [nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data] Package treebank is already up-to-date!
Dataset loaded successfully!
Total sentences: 3914

HMM Model trained successfully!

Labeled Sentence:
[('Natural', 'NNP'), ('Language', 'NNP'), ('Processing', 'NNP'), ('is', 'NNP'), ('fascinating', 'NNP')]
```

Fig 5: Dataset used for training or testing

```
[90] from nltk.corpus import treebank
✓ 0s print(treebank.tagged_sents()[0])
... [('Pierre', 'NNP'), ('Vinken', 'NNP'), (',', ','), ('61', 'CO'), ('years', 'NNS'), ('old', 'JJ'), (',', ','), ('will', 'MD'), ('join', 'VB'), ('the', 'DT'), ('board', 'NN'), ('as', 'IN'), ('a', 'DT'), ('nonexecutive', 'NNP')]
```

REFLECTION

Hidden Markov Models (HMMs) are probabilistic models used for sequence labeling tasks where the system predicts hidden states based on observed data. In this exercise, HMMs were applied to part-of-speech tagging and sentence labeling using the Treebank dataset. Sequence labeling assigns tags to words based on context and probability transitions between states. Named Entity Recognition demonstrated how words are classified into entities such as persons and locations. The results showed that HMMs are effective for structured language tasks but may struggle with unseen words. This highlights their importance in NLP applications like speech recognition, text classification, and information extraction systems.

WEEK 4: SYNTACTIC AND SEMANTIC ANALYSIS

Syntactic Analysis, also called parsing, helps understand how words are arranged in a sentence according to grammar rules. It identifies relationships between words and creates a parse tree representing the sentence structure.

Example:

Example

Consider the following sentences

- Correct Syntax: "John eats an apple."
- Incorrect Syntax: "Apple eats John an."

Although both sentences contain the same words, only the first sentence follows correct grammatical structure and conveys meaningful information.

Semantic analysis focuses on understanding the meaning of words and sentences. It ensures that text is not only grammatically correct but also logically meaningful and contextually appropriate.

Example:

Example

Sentence: "Apple eats a John."

Although the sentence is grammatically correct, it is semantically incorrect because an apple cannot perform the action of eating a person.

PRACTICAL TASK 1

Dependency Parsing Using spaCy

To Analyze grammatical structure of a sentence.

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "The lecturer teaches Natural Language Processing."

doc = nlp(text)

for token in doc:
    print(token.text, token.dep_, token.head.text)

... The det lecturer
    lecturer nsubj teaches
    teaches ROOT teaches
    Natural compound Language
    Language compound Processing
    Processing dobj teaches
    . punct teaches
```

The figure above shows an example of a code that demonstrates dependency parsing.

PRACTICAL TASK 2

Semantic Similarity Analysis

Objective

Compare meaning similarity between sentences.

```
import spacy

nlp = spacy.load("en_core_web_md")

sentence1 = "I love eating apples"
sentence2 = "I enjoy consuming fruits"

doc1 = nlp(sentence1)
doc2 = nlp(sentence2)

similarity_score = doc1.similarity(doc2)

print(f"Sentence 1: {sentence1}")
print(f"Sentence 2: {sentence2}")
print(f"Semantic similarity: {similarity_score:.3f}")

... Sentence 1: I love eating apples
Sentence 2: I enjoy consuming fruits
Semantic similarity: 0.811
```

The figure above shows a code that demonstrates similarity between sentences.

CLASS ASSIGNMENT

Assignment 1

Analyze dependency structure for:

1. "The administrator updated student records"
2. "Machine learning improves language processing"

Sentence 1: “The administrator updated student records”.

```
import spacy

nlp = spacy.load("en_core_web_sm")

text1 = "The administrator updated student records"
doc1 = nlp(text1)

print("Sentence 1:", text1)
print("-" * 40)
for token in doc1:
    print(f"{token.text:15} {token.dep_:12} {token.head.text:15}")
```

```
*** Sentence 1: The administrator updated student records
-----
The          det          administrator
administrator nsubj      updated
updated      ROOT        updated
student      compound     records
records      dobj        updated
```

In the sentence "**The administrator updated student records**," the verb "**updated**" is the root and main action. "**Administrator**" is the subject (**nsubj**) performing the action, while "**The**" is a determiner (**det**) describing the subject. "**Records**" is the direct object (**dobj**) receiving action, and "**student**" is a compound modifier (**compound**) specifying the type of records. The sentence follows a clear **subject–verb–object** structure.

Sentence 2: “Machine learning improves language processing”

```
text2 = "Machine learning improves language processing"
doc2 = nlp(text2)

print("\nSentence 2:", text2)
print("-" * 40)
for token in doc2:
    print(f"{token.text:15} {token.dep_:12} {token.head.text:15}")

...
Sentence 2: Machine learning improves language processing
-----
Machine      compound    learning
learning     nsubj       improves
improves     ROOT        improves
language     compound    processing
processing   dobj        improves
```

In the sentence "**Machine learning improves language processing**," the verb "**improves**" is the root and main action. "**Machine learning**" acts as the subject (**nsubj**) performing the action, where "**Machine**" modifies "**learning**" as a compound noun. Similarly, "**language processing**" is the object receiving the action, with "**language**" modifying "**processing**" as a compound noun. This sentence follows a subject–verb–object structure, showing that machine learning improves language processing.

Assignment 2

Create semantic similarity comparison for:

- Two similar sentences
- Two unrelated sentences

```

import spacy

nlp = spacy.load("en_core_web_md")

similar1 = "The cat sleeps on the sofa"
similar2 = "The feline rests on the couch"

unrelated1 = "The cat sleeps on the sofa"
unrelated2 = "The stock market crashed today"

doc_sim1 = nlp(similar1)
doc_sim2 = nlp(similar2)
doc_unrel1 = nlp(unrelated1)
doc_unrel2 = nlp(unrelated2)

similarity_score = doc_sim1.similarity(doc_sim2)
unrelated_score = doc_unrel1.similarity(doc_unrel2)

print("SEMANTIC SIMILARITY COMPARISON")

print("\nPair 1 - SIMILAR Sentences:")
print(f"  A: '{similar1}'")
print(f"  B: '{similar2}'")
print(f"  Similarity Score: {similarity_score:.3f}")

print("\nPair 2 - UNRELATED Sentences:")
print(f"  A: '{unrelated1}'")
print(f"  B: '{unrelated2}'")
print(f"  Similarity Score: {unrelated_score:.3f}")

print("RESULTS:")

```

```

... SEMANTIC SIMILARITY COMPARISON

```

```

Pair 1 - SIMILAR Sentences:
  A: 'The cat sleeps on the sofa'
  B: 'The feline rests on the couch'
  Similarity Score: 0.806

Pair 2 - UNRELATED Sentences:
  A: 'The cat sleeps on the sofa'
  B: 'The stock market crashed today'
  Similarity Score: 0.569
RESULTS:

```

Pair one has higher similarities compared to pair two

Reasons why:

The results differ because of semantic overlap. The similar pair shares synonyms like "cat/feline" and "sofa/couch," giving them nearly identical word vectors and a high cosine similarity (0.806).

The unrelated pair has no conceptual connection between animals vs. Finance, so their vectors point in opposite directions, yielding a low score.

PRACTICAL TASK 1

Complete NLP Pipeline

Objective

Build a simple NLP processing pipeline.

```
import nltk

nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger_eng')

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag

text = "Natural Language Processing helps computers understand language."

words = word_tokenize(text)
print("Tokens:", words)

stop_words = set(stopwords.words('english'))
filtered = [w for w in words if w.lower() not in stop_words]
print("Filtered Words:", filtered)

pos = pos_tag(filtered)
print("POS Tags:", pos)

... [nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
Tokens: ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'language', '.']
Filtered Words: ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'language', '.']
POS Tags: [('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('helps', 'VBZ'), ('computers', 'NNS'), ('understand', 'JJ'), ('language', 'NN'), ('.', '.')]

```

PRACTICAL TASK 2

Mini Chatbot Simulation

Objective

Create a simple rule-based chatbot.

```
print("Welcome to Student Help Chatbot")

while True:
    user = input("You: ")

    if "hello" in user.lower():
        print("Bot: Hello Student")

    elif "exam" in user.lower():
        print("Bot: CAT 1 starts next week")

    elif "registration" in user.lower():
        print("Bot: Unit registration is open")

    elif "bye" in user.lower():
        print("Bot: Goodbye")
        break

    else:
        print("Bot: I do not understand")
```

```
Welcome to Student Help Chatbot
You: hello
Bot: Hello Student
```


WEEK 5 MINI PROJECT

Project Title

Student Academic Assistant Chatbot

```
print("STUDENT ACADEMIC ASSISTANT CHATBOT")
print("\nType 'bye' to exit\n")

while True:
    user = input("\nYou: ").lower()

    # Greetings
    if user in ["hello", "hi", "hey"]:
        print("Bot: Hello Student! How can I help you today?")

    # CAT Schedule Questions
    elif "cat" in user or "exam" in user:
        print("Bot: CAT 1 starts next week. Please check the class timetable for exact dates.")

    # Unit Registration Questions
    elif "registration" in user or "register" in user:
        print("Bot: Unit registration is currently open. Please use the student portal to register your units.")

    # Exit Command
    elif user == "bye":
        print("Bot: Goodbye! Have a great day.")
        break

    # Unknown Input
    else:
        print("Bot: Sorry, I do not understand your question.")
```

Interactions with the chatbot:

```
... STUDENT ACADEMIC ASSISTANT CHATBOT
Type 'bye' to exit

You: hello
Bot: Hello Student! How can I help you today?

You: cat
Bot: CAT 1 starts next week. Please check the class timetable for exact dates.

You: unit registration
Bot: Unit registration is currently open. Please use the student portal to register your units.

You: bye
Bot: Goodbye! Have a great day.
```

Challenge:

This chatbot is rule-based and can only respond to predefined keywords. More advanced chatbots use machine learning, NLP models, and Large Language Models (LLMs) to understand context and generate intelligent responses.

REVISION AREAS

1.Tokenization is the process of breaking text into smaller units called tokens, which helps machines process and analyze language effectively.

2.Parts of Speech (PoS) tagging is a fundamental task in Natural Language Processing (NLP) where each word in a sentence is assigned a grammatical category such as noun, verb, adjective or adverb. This process helps machines understand the structure and meaning of sentences by identifying the roles of words and their relationships.

3. N-gram is a language modelling technique that is defined as the contiguous sequence of n items from a given sample of text or speech. The N-grams are collected from a text or speech corpus.